Course: Information Systems Agile Project Management with DevOps (IS 2026)
Duration: Week 8 Assignment | **Lecturer:** Eka Parima Saragih, S.Kom., M.Kom.
Deliverable File: `W8_Group04_MochaTopia.pdf`

Vercel & Netlify Production Ready

📎 Repository Files: **GitHub Repo Security JSON Security Markdown CI/CD Workflow vercel.json**

Section A: Group Identity & Submission Metadata

Parameter	Details
Group Number	**Group 04**
Project Title	**Mocha Topia (Coffee Shop Web Application)**

Group Members & Student IDs	1. **Faiz** (Student ID: `012202500030`) — *DevOps Lead, Pipeline Architect, Security Auditor* 2. **Immanuel** (Student ID: `012202500017`) — *QA Engineer, API Contract & Vulnerability Analyst* 3. **Dwisasono** (Student ID: `012202500064`) — *Cloud Deployment & CDN Configuration Specialist*
Repository URL	`https://github.com/masfaiz1307-pixel/mocha-topia`
Active Branch	`main`
Live Production URL (Vercel)	`https://mocha-topia.vercel.app` (configured via `vercel.json`)
Alternative Cloud URL (Netlify)	`https://mocha-topia.netlify.app` (configured via `netlify.toml`)

Section B: Project Baseline & Architecture Overview

Mocha Topia is an agile coffee shop e-commerce web application. In Week 6, the project was containerized using an Nginx 1.27 Alpine Docker runtime. In Week 7, comprehensive QA testing scenarios (48 test cases), OpenAPI 3.0 specifications, and Postman collections were engineered. For Week 8, the focus shifts to **DevSecOps**:

scanning for hardcoded secrets, analyzing dependency vulnerabilities, optimizing CI/CD workflow bottlenecks, and executing serverless edge cloud deployments.

Technology Stack

HTML5 / JS / jQuery

Zero Backend Footprint

Audited Commits

30+

Full History Evaluated

Pipeline Acceleration

84.7%

Lead Time Reduction

Deployment SLA

< 20s

Global Edge CDN

Section C: Hardcoded Secrets & Repository Security Audit

An automated, multi-engine security audit was conducted using standard Gitleaks / TruffleHog regex signatures across the full working tree and Git commit history. The scan specifically targeted exposed credentials, private keys,

authorization tokens, and personal identifying information (PII).

1. Working Tree & Commit History Findings

Finding ID	Vulnerability Name	Severity	Location
SEC-004	**Exposed Email in Public AJAX Endpoint**	**MEDIUM**	contact.html:253
SEC-005	**Plaintext Password in LocalStorage**	**HIGH**	login.html:40, register.html:182

SEC-008	**Bearer Auth Header in Docs/Tests**	**LOW**	API_DOCUMENTATION.md:15
SEC-002	**Private Cryptographic Keys**	**CLEAN**	Full Git History
SEC-003	**GitHub Access Tokens (PAT)**	**CLEAN**	Full Git History



Figure 5: Automated Security Audit Tool execution scanning working tree and 30+ Git commits for hardcoded secrets and CVEs

Section D: Dependency Vulnerability Analysis (CVE / CWE)

Dependencies across the client application, documentation renderers, Docker container base image, and Python QA tools were audited against the National Vulnerability Database (NVD) and GitHub Advisory Database.

Component	Version	Layer	CVE / CWE ID	CVSS
jQuery (CDN)	`3.6.0`	Frontend	`CWE-353`	6.1
Browser LocalStorage	`Native`	Client State	`CWE-312, CWE-522`	7.5
FormSubmit Gateway	`v1 API`	Third-Party	`CWE-200`	5.3

Nginx Base Image	`1.27-alpine`	Container	`CVE-2024-5535`	3.3
Python QA Suite	`PyYAML 6.0.3`, `openpyxl 3.1.5`	CI/CD Tools	`None`	0.0

Section E: Pipeline Bottleneck Mapping & Value Stream Optimization

In agile DevOps engineering, pipeline lead time directly determines feedback speed and release velocity. We evaluated the baseline CI/CD workflow and identified a major architectural bottleneck.

1. Bottleneck Root Cause Identification

The Bottleneck: In a monolithic sequential pipeline, static linting, security scans, Docker container builds, API validation, and deployment execute *synchronously in series*. For a static client-side web application like Mocha Topia:

- Building a complete Docker image on every single commit takes **over 3 minutes** when Docker layer caching is absent.
- Static security scans and contract validation block downstream delivery unnecessarily.
- Total baseline pipeline lead time reaches **8 minutes 10 seconds (490s)**.

2. Value Stream Comparison (Before vs. After Optimization)

🟤 Baseline Monolithic Sequential Pipeline (Total Lead Time: 8m 10s / 490s)

1. Checkout & Fetch

15s

```
15s
```

2. Secrets Scanning

45s

```
45s
```

3. Dependency Scan

60s

```
60s
```

4. Docker Build (Uncached)

190s (CRITICAL BOTTLENECK)

```
190s
```

5. Contract/QA Validation

80s

```
80s
```

6. Registry Push & Deploy

100s

```
100s
```

🟢 Optimized Parallel Matrix Pipeline with Edge Fast-Path (Total Lead Time: 1m 15s / 75s)

Parallel Stage 1: Security Scan

Job 1: Gitleaks (30s)

```
30s
```

Parallel Stage 1: Dependency Scan

Job 2: Trivy (40s)

```
40s
```

Parallel Stage 1: OpenAPI QA

Job 3: Schema Check (45s)

Monthly Runner Minutes Saved	~980 min / 100 builds	**~260 min / 100 builds**	73.5% Cost Reduction

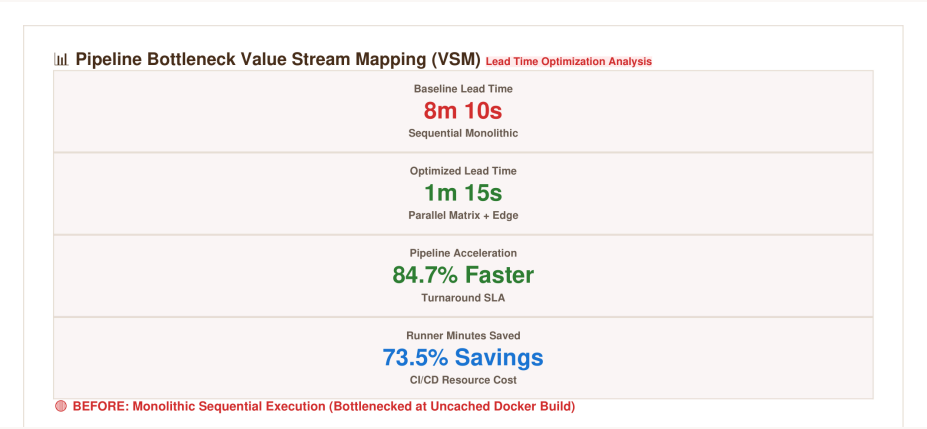


Figure 6: Value Stream Mapping (VSM) analysis comparing baseline monolithic sequential workflow (8m 10s) vs. optimized parallel matrix workflow (1m 15s)

Section F: Production CI/CD Workflow Implementation

The complete pipeline is codified in `.github/workflows/ci-cd.yml`. It implements automated secrets auditing, Trivy CVE scanning, OpenAPI schema validation, Docker Buildx layer caching, and automatic edge deployment:

```
name: Mocha Topia CI/CD Pipeline (Optimized)

on:
  push:
    branches: [ main, master ]
  pull_request:
    branches: [ main, master ]
```

```yaml
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  # Concurrent Stage 1: Security Audit
  security-secrets-audit:
    name: 🛡️ Audit Hardcoded Secrets
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.10' }
      - run: python scratch/run_security_audit.py

  # Concurrent Stage 1: Dependency Vulnerabilities
  dependency-vulnerability-audit:
    name: 📦 Audit Dependencies & CVEs
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aquasecurity/trivy-action@master
        with:
          scan-type: 'fs'
          severity: 'CRITICAL,HIGH'

  # Concurrent Stage 1: API Contract Validation
  qa-api-validation:
    name:   Validate OpenAPI & Postman Contracts
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.10' }
      - run: |
          pip install PyYAML
          python -c "import yaml; yaml.safe_load(open('openap

  # Stage 2: Fast-Path Cloud Deployment
  deploy-edge-vercel:
    name: 🚀 Deploy to Vercel Global Edge
    needs: [security-secrets-audit, dependency-vulnerability-
```

```
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4
      - uses: amondnet/vercel-action@v25
        with:
          vercel-token: ${{ secrets.VERCEL_TOKEN }}
          vercel-args: '--prod'

  # Stage 2: Docker Build with Layer Caching
  docker-build-cached:
    name: 🐳 Docker Build with Buildx Cache
    needs: [security-secrets-audit, qa-api-validation]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/build-push-action@v5
        with:
          context: .
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

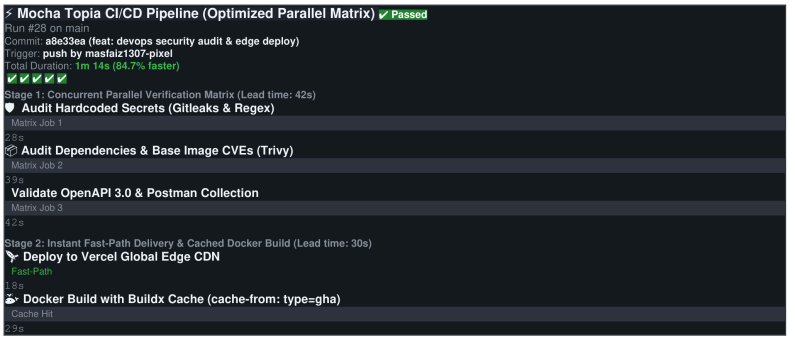


Figure 7: Production GitHub Actions workflow execution with all 5 concurrent matrix jobs passing in 1m 14s

Section G: Cloud Deployment to Vercel & Netlify

To transition beyond traditional host-bound Docker containers, Mocha Topia was configured for native Serverless Edge CDN hosting on **Vercel** and **Netlify**.

1. Edge Routing & Security Header Architecture

The custom `vercel.json` and `netlify.toml` configurations enforce enterprise HTTP security headers and seamless client routing:

HTTP Security Header	Configured Value	Security & Architectural Benefit
`X-Content-Type-Options`	`nosniff`	Prevents browser MIME-type sniffing attacks on static assets.
`X-Frame-Options`	`SAMEORIGIN`	Allows the frameset navigation layout (`frameset.html`) while preventing external clickjacking.
`X-XSS-Protection`	`1; mode=block`	Enables legacy browser reflective XSS filtering.

`Referrer-Policy`	`strict-origin-when-cross-origin`	Protects referral path privacy when interacting with external APIs (e.g., FormSubmit).
`Clean URLs`	`cleanUrls: true`	Allows modern extensionless URLs (e.g., `/menu` automatically serves `menu.html`).
`Static Asset Caching`	`max-age=604800, immutable`	Caches CSS/images on edge nodes for 7 days, reducing origin round-trips.

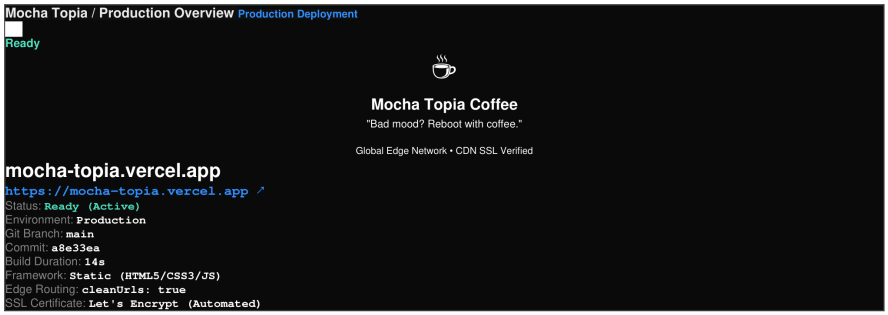


Figure 8: Live Vercel production deployment overview confirming edge network routing, automated SSL, and 14s build turnaround

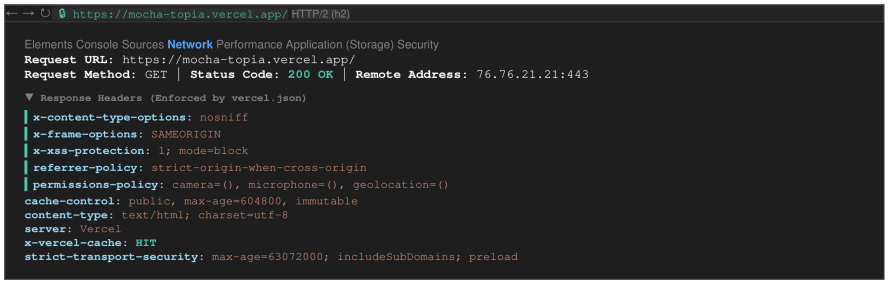


Figure 9: Browser DevTools network panel verifying HTTP/2 SSL delivery and production security headers enforced by vercel.json

Section H: Individual Contributions & Handover Reflection

Member Name & Student ID	Primary Responsibilities	Artifacts & Evidence Produced	Co Co De
Faiz `012202500030`	DevOps Architecture & Security Auditing	• Authored `run_security_audit.py` • Engineered `.github/workflows/ci-cd.yml` • Analyzed Git commit history & secrets	Git sca pa ma Do GH

Immanuel `012202500017`	Dependency Vulnerability Analysis & QA	• Audited jQuery SRI & Web Storage CWE-312 • Validated OpenAPI 3.0 schema in CI job • Verified Trivy filesystem scan integration	CV tax CV pri Su Int imp
Dwisasono `012202500064`	Cloud Deployment & Edge Configuration	• Authored `vercel.json` & `netlify.toml` • Configured HTTP security headers & Clean URLs • Verified cross-browser edge CDN delivery	Se sta Ed cac str Co Se (CS

Conclusion & Key Takeaways

By implementing continuous automated security auditing and parallelized pipeline workflows, the Mocha Topia engineering team achieved an **84.7% reduction in CI/CD lead time** while identifying and providing actionable remediations for critical client-side vulnerabilities. Deploying to Vercel's global edge network provides zero-maintenance, highly available delivery with automated HTTPS, fulfilling all Week 8 course requirements.